

Apunte de la Fase 3

Núcleo algorítmico, eficiencia y programación orientada a objetos

El cambio de foco. Las Fases 1 y 2 tenían por objeto los datos: su origen, su calidad y su preparación. La Fase 3 tiene por objeto el programa que los procesa: su estructura, su costo computacional y su capacidad de evolucionar. No se cambia de conjunto ni se rehace el pipeline. Se reorganiza lo que ya existe bajo criterios de ingeniería de software.

1. Marco conceptual: tres paradigmas que se integran

Un paradigma de programación es un modo de organizar la solución de un problema computacional. No son excluyentes: un mismo programa puede combinarlos, y eso es justamente lo que se pide en esta fase.

Paradigma	Unidad de organización	Qué aporta al proyecto
Estructurado	La función, con sus estructuras de control	Descompone el problema en tareas con entrada y salida definidas
Recursivo	La función que se invoca a sí misma	Resuelve problemas cuya profundidad no se conoce de antemano
Orientado a objetos	El objeto, que reúne estado y comportamiento	Agrupar el dato con el código que lo usa, y permite extender sin modificar

La secuencia tiene sentido histórico y pedagógico. La programación estructurada surge para eliminar el salto incondicional y hacer el flujo predecible. La recursividad aporta una forma de expresar problemas autosimilares. Y la orientación a objetos responde a un problema que las dos anteriores no resuelven bien: cuando el estado de un cálculo debe sobrevivir entre invocaciones, las variables globales se vuelven inmanejables.

Los dos conceptos raíz

Abstracción. Exponer qué hace un componente y ocultar cómo lo hace. Quien usa una función de imputación necesita saber qué recibe y qué devuelve, no el algoritmo interno.

Modularidad. Dividir el sistema en partes con responsabilidades delimitadas, de modo que cada una se pueda entender, probar y modificar por separado.

Todo lo demás de este apunte deriva de esos dos. El encapsulamiento es abstracción aplicada al estado. Los patrones de diseño son formas conocidas de lograr modularidad. Y el criterio de cohesión mide cuán bien se hizo la división.

2. Programación estructurada aplicada al pipeline

La descomposición funcional consiste en separar un procedimiento extenso en funciones con un propósito único, un contrato explícito de entrada y salida, y sin dependencias ocultas del orden de ejecución.

Principio de responsabilidad única. Un componente debe tener una sola razón para cambiar. Si una función se modifica tanto cuando cambia la fuente de datos como cuando cambia la regla de limpieza, está haciendo dos cosas.

Una señal práctica de violación de este principio es la función con parámetros de control booleanos. Si un procedimiento recibe tres banderas y ejecuta ramas distintas según su valor, en realidad contiene varias funciones que conviene separar.

La otra señal es el acoplamiento por orden implícito: celdas que solo funcionan si se ejecutaron las anteriores en cierta secuencia, sin que esa secuencia esté declarada en ninguna parte.

3. Recursividad

Una función recursiva resuelve un problema reduciéndolo a una instancia menor del mismo problema. Su corrección depende de dos condiciones que deben verificarse siempre.

Elemento	Qué garantiza
Caso base	Que la recursión termine. Define la instancia mínima que se resuelve de forma directa.
Caso recursivo	Que cada llamada se acerque al caso base. Si el problema no decrece, la recursión no termina.

Qué ocurre en memoria

Cada invocación pendiente ocupa un marco en la pila de llamadas, con sus variables locales y su punto de retorno. De ahí se derivan dos consecuencias. La primera es que la recursión tiene un costo de espacio proporcional a la profundidad. La segunda es que Python impone un límite de profundidad, cercano a mil llamadas por defecto, y al superarlo interrumpe la ejecución.

```
def aplanar(estructura, prefijo=""):
    """Convierte una estructura anidada en pares de clave y valor."""
    plano = {}
    for clave, valor in estructura.items():
        compuesta = f"{prefijo}.{clave}" if prefijo else str(clave)
        if isinstance(valor, dict) and valor:
            plano.update(aplanar(valor, compuesta)) # caso recursivo
        else:
            plano[compuesta] = valor # caso base
    return plano
```

Recursión frente a iteración

La recursión se justifica cuando	La iteración es preferible cuando
La profundidad del problema no se conoce al escribir el código. La estructura es autosimilar: cada nivel se trata igual que el anterior. El algoritmo sigue una estrategia de división y conquista.	La profundidad es fija y conocida. El volumen puede agotar la pila. La versión iterativa es más legible y no requiere estructuras auxiliares.

Para el informe. Si el proyecto no contiene un problema recursivo natural, corresponde declararlo y argumentar por qué el enfoque iterativo es preferible. El descriptor admite recursividad o división funcional: lo que se evalúa es el criterio, no la técnica.

Recursión y eficiencia no son lo mismo

Una recursión que recalcula subproblemas ya resueltos tiene costo exponencial. La misma recursión, guardando los resultados intermedios en un diccionario, pasa a costo lineal. La técnica se llama memoización y consiste en cambiar tiempo por espacio: se almacena para no recalcular.

4. Complejidad algorítmica

La complejidad describe cómo crece el costo de un algoritmo cuando crece el tamaño de la entrada. No mide segundos: mide la forma de la curva. Por eso es independiente del computador donde se ejecute.

Notación asintótica. La expresión $O(f(n))$ indica que, para entradas suficientemente grandes, el número de operaciones crece a lo más proporcionalmente a $f(n)$. Se ignoran las constantes y los términos de menor orden, porque no cambian la forma del crecimiento.

Orden	Nombre	Ejemplo en un pipeline de datos
$O(1)$	Constante	Buscar una clave en un diccionario
$O(\log n)$	Logarítmico	Búsqueda binaria sobre una columna ordenada
$O(n)$	Lineal	Recorrer todas las filas una vez
$O(n \log n)$	Cuasilineal	Ordenar el conjunto por una columna
$O(n^2)$	Cuadrático	Comparar cada fila con todas las demás
$O(2^n)$	Exponencial	Recursión sin memoria sobre subproblemas repetidos

Complejidad temporal y espacial

La temporal cuenta operaciones; la espacial cuenta memoria adicional. Con frecuencia se contraponen: la memoización reduce el tiempo aumentando el espacio, y la lectura por bloques reduce el espacio aumentando el tiempo. Una justificación completa considera ambas dimensiones, y el descriptor de la rúbrica lo exige expresamente.

Análisis asintótico y medición empírica

El análisis asintótico predice la forma del crecimiento; la medición empírica entrega el costo real en un equipo concreto. Ambos son necesarios y ninguno reemplaza al otro. Un algoritmo $O(n)$ puede ser más lento que uno $O(n^2)$ en entradas pequeñas, porque las constantes que el análisis ignora sí pesan cuando n es chico.

Consecuencia práctica. Una medición sobre un solo tamaño no permite inferir la complejidad. Para observar la forma de la curva hay que medir sobre tamaños crecientes y comparar las razones entre implementaciones.

Instrumentos de medición

Herramienta	Qué mide	Cuándo usarla
<code>time.perf_counter()</code>	Tiempo transcurrido de un intervalo, con alta resolución	Etapas del pipeline
<code>timeit.timeit()</code>	Tiempo acumulado repitiendo la operación	Fragmentos breves
<code>tracemalloc</code>	Memoria asignada por el intérprete	Conjuntos grandes

```
import timeit

t_a = timeit.timeit(lambda: version_a(df), number=20)
t_b = timeit.timeit(lambda: version_b(df), number=20)
print(f"A: {t_a:.4f} s   B: {t_b:.4f} s   razón: {t_a / t_b:.1f}")

assert version_a(df) == version_b(df), "Las versiones no coinciden"
```

Condiciones de validez de una medición

1. **Repetir y conservar el mínimo.** Los valores altos reflejan interferencia del sistema operativo, no el costo del código. El mínimo es el estimador más cercano al costo real.
2. **Medir sobre el tamaño de trabajo.** En entradas pequeñas dominan los costos fijos y la comparación no discrimina.
3. **Verificar equivalencia funcional.** Dos implementaciones solo son comparables si producen el mismo resultado.

5. Programación orientada a objetos

Objeto. Entidad que reúne un **estado**, los datos que recuerda, y un **comportamiento**, las operaciones que sabe ejecutar sobre ese estado.

Clase. Descripción de la estructura y el comportamiento común a un conjunto de objetos. Define un tipo; no es un dato ni ocupa estado propio.

La instanciación es el acto de crear un objeto a partir de una clase. En Python la disparan los paréntesis, que provocan la reserva de memoria y la ejecución automática del constructor `__init__`. El parámetro `self` es la referencia al objeto sobre el que se opera: la invocación `obj.metodo(x)` equivale a `Clase.metodo(obj, x)`.

Criterio para introducir una clase. Si el componente debe conservar estado entre invocaciones, la clase se justifica. Si cada invocación es independiente, una función es más simple y más fácil de verificar. Señal equivalente: si una función requiere una variable global para operar, esa variable debería ser un atributo.

Los cuatro pilares

Abstracción

La clase expone una interfaz estable, el conjunto de métodos públicos, y reserva la implementación. Quien usa un transformador invoca `ajustar` y `transformar` sin conocer el cálculo interno. Cambiar la implementación no obliga a cambiar a

quien la usa.

Encapsulamiento

Es la ocultación deliberada del estado interno y el control de acceso a él. En Python se señala con un guion bajo inicial, que es una convención respetada por las herramientas, y se refuerza con validaciones que impiden estados inconsistentes. Su efecto práctico es convertir un error silencioso en un error visible y localizado en su origen.

Herencia

Relación entre una clase base y clases derivadas que reutilizan su estructura y comportamiento. La clase base define el contrato; cada derivada implementa lo que la distingue. La resolución de un atributo sigue una cadena de búsqueda: primero el objeto, luego su clase, luego las clases ascendientes.

Herencia frente a composición. La herencia expresa una relación de especialización, «es un». La composición expresa una relación de uso, «tiene un». Cuando la relación no es de especialización, la composición produce diseños más flexibles: el patrón Strategy es composición, no herencia.

Polimorfismo

Capacidad de invocar la misma operación sobre objetos de tipos distintos, obteniendo el comportamiento propio de cada uno. Requiere que las clases compartan interfaz. Su valor no es estético: permite incorporar un tipo nuevo sin modificar el código que lo consume.

Principio de sustitución. Un objeto de una clase derivada debe poder reemplazar a uno de su clase base sin alterar la corrección del programa. Si una derivada exige condiciones adicionales o incumple lo que la base promete, el polimorfismo deja de ser seguro.

6. La estructura que hay que construir

```
class Transformador:
    """Clase base: define el contrato común de todos los pasos."""

    def __init__(self, columna):
        self.columna = columna
        self._parametros = {}          # estado interno: lo aprendido
        self._ajustado = False         # estado interno: control de orden

    def ajustar(self, df):
        if self.columna not in df.columns:
            raise KeyError(f"La columna {self.columna} no existe.")
        self._parametros = self.aprender(df)
        self._ajustado = True
        return self

    def transformar(self, df):
        if not self._ajustado:
            raise RuntimeError("Hay que ajustar antes de transformar.")
        return self.aplicar(df.copy())  # copia: no se muta la entrada

    def aprender(self, df):
        raise NotImplementedError("Cada clase derivada debe implementarlo.")

    def aplicar(self, df):
        raise NotImplementedError("Cada clase derivada debe implementarlo.")

class ImputadorMediana(Transformador):          # herencia

    def aprender(self, df):
        return {"mediana": float(df[self.columna].median())}

    def aplicar(self, df):
        df[self.columna] = df[self.columna].fillna(self._parametros["mediana"])
        return df
```

Esta clase base aplica el **método plantilla**: define el esqueleto del procedimiento, con sus validaciones y su control de estado, y delega a las clases derivadas los pasos que varían. Los métodos que lanzan `NotImplementedError` hacen explícito el contrato: una derivada que los omita falla con un mensaje que indica la causa.

```
class Pipeline:
    def __init__(self, pasos):
        self._pasos = list(pasos)
        self._ajustado = False

    def ajustar(self, df):
        intermedio = df.copy()
        for paso in self._pasos:
            # polimorfismo
            intermedio = paso.ajustar(intermedio).transformar(intermedio)
        self._ajustado = True
        return self

    def transformar(self, df):
        if not self._ajustado:
            raise RuntimeError("Hay que ajustar antes de transformar.")
        resultado = df.copy()
        for paso in self._pasos:
            resultado = paso.transformar(resultado)
        return resultado
```

El pipeline es composición: contiene transformadores y los coordina, sin heredar de ellos ni conocer su tipo concreto. El bucle no contiene ninguna condición sobre el tipo de paso, y esa ausencia es la evidencia del polimorfismo.

Por qué se separa ajustar de transformar. Los parámetros se estiman únicamente con el conjunto de entrenamiento y se aplican al de prueba. Estimarlos sobre el conjunto completo introduce en el modelo información que no debía estar disponible. Esa contaminación se denomina fuga de datos y compromete la validez de toda evaluación posterior.

Extensión de una clase derivada

```
class ImputadorFlexible(Transformador):
    def __init__(self, columna, estrategia=None):
        super().__init__(columna) # ejecuta el constructor de la base
        self.estrategia = estrategia # y agrega su propio estado
```

Al definir un constructor propio, la clase derivada reemplaza al de la base. La invocación de `super()` restituye esa ejecución. Omitirla deja el objeto sin los atributos heredados, y el error se manifiesta más tarde, lejos de su causa.

7. Principios de diseño

Cohesión. Grado en que los elementos de un módulo contribuyen a un único propósito. Es deseable que sea alta. Señal de cohesión baja: describir el módulo requiere la conjunción «y» varias veces.

Acoplamiento. Grado de dependencia entre módulos. Es deseable que sea bajo. Se reduce haciendo que los módulos dependan de interfaces y no de implementaciones concretas.

Los dos criterios operan en conjunto y se orientan al mismo objetivo: limitar la propagación de un cambio. Un sistema con alta cohesión y bajo acoplamiento permite modificar una decisión sin revisar el resto.

Prueba operacional del acoplamiento. ¿Cuántos archivos hay que modificar para cambiar la estrategia de imputación? Si la respuesta es uno, el acoplamiento es bajo. Esa formulación, con el número, sostiene la justificación de la arquitectura mejor que una definición general.

Principio de abierto y cerrado. Un módulo debe estar abierto a la extensión y cerrado a la modificación: incorporar un comportamiento nuevo no debería exigir alterar el código existente. El pipeline lo cumple: agregar un transformador no obliga a modificar el bucle.

8. Patrones de diseño

Patrón de diseño. Solución documentada y reutilizable a un problema recurrente de diseño, descrita de forma independiente del lenguaje. No es código para copiar: es una estructura de relaciones entre componentes.

La clasificación clásica distingue tres familias según el problema que atienden.

Familia	Problema que atiende	Patrones de esta fase
Creacionales	Cómo se crean los objetos, desacoplando al cliente de la clase concreta	Factory, Singleton
Estructurales	Cómo se componen los objetos en estructuras mayores	No se abordan en esta fase
De comportamiento	Cómo se distribuyen las responsabilidades y la comunicación	Strategy, Observer

Patrón	Estructura y propósito	Manifestación en el proyecto
Strategy	Encapsula algoritmos intercambiables tras una interfaz común y los inyecta por composición	Comparar estrategias de imputación sin modificar la clase que imputa
Factory	Centraliza la decisión de qué clase concreta instanciar según una condición	Construir el transformador que corresponde al rol de cada variable
Observer	Define una dependencia de uno a varios: el sujeto notifica a observadores que no conoce	La bitácora se genera durante la ejecución y no se redacta después
Singleton	Restringe una clase a una única instancia con punto de acceso global	Configuración compartida: semilla, rutas y umbrales

Criterio de aplicación. Un patrón se justifica cuando existe el problema que resuelve. Aplicado sin ese problema incorpora complejidad sin beneficio, y ese exceso también se evalúa. Singleton merece cautela adicional: introduce estado global, que dificulta las pruebas.

Cómo se documenta un patrón

Nombrarlo no basta. La documentación pertinente declara tres elementos: el problema concreto que existía, la forma en que el patrón lo resuelve y la consecuencia de no haberlo aplicado. Por ejemplo: «Se aplicó Strategy en la imputación. Antes, cambiar de mediana a moda exigía editar la función. Con la estrategia inyectada como parámetro la clase permanece intacta, lo que permitió comparar tres alternativas sin reescribir código».

9. Aplicación a la Fase 3

Entregables

Producto	Formato	Contenido
Informe	PDF	Los apartados de la plantilla institucional
Notebook	.ipynb	Entregable central: código, mediciones y clases
Repositorio	Enlace	Carpeta F2 referencial y carpeta F3 con el avance

La **Formativa 3** no pondera y no exige clases: pide scripts estructurados y mediciones de complejidad. La **Sumativa 2** pondera el 20 por ciento y sí exige herencia, polimorfismo, encapsulamiento y al menos un patrón. De sus 69 puntos, 45 corresponden al notebook y el código, 18 al informe y 6 al repositorio.

Secuencia de trabajo

1. Inventariar los pasos del pipeline de la Fase 2 e identificar los que comparten estructura.
2. Definir la clase base con el contrato común: validación, control de estado y copia defensiva.
3. Implementar dos o tres clases derivadas, una por tipo de transformación.

4. Componer el pipeline que las recorre mediante la interfaz común.
5. Verificar equivalencia con el resultado de la Fase 2.
6. Medir y comparar al menos dos implementaciones de un mismo cálculo.
7. Seleccionar un patrón, implementarlo y documentar la arquitectura resultante.
8. Redactar el informe declarando dónde se manifiesta cada principio.

La verificación de mayor rendimiento. Contrastar el conjunto procesado de la Fase 2 con el que produce el pipeline de clases mediante `pd.testing.assert_frame_equal`. La coincidencia constituye evidencia objetiva de que la reorganización preservó la semántica. La discrepancia revela un defecto antes de la entrega.

Contrato con la Fase 4

Artefacto	Función en la fase siguiente
Conjunto de entrenamiento procesado	Exploración y análisis
Conjunto de prueba procesado	Evaluación, con esquema idéntico
Diccionario del resultado	Interpretación de cada variable y su origen
Parámetros aprendidos por cada paso	Reproducción y auditoría del resultado

La Fase 4 no debe reaplicar imputación, codificación ni escalamiento. Sobre un conjunto ya transformado, el escalamiento se aplicaría dos veces y el resultado perdería sentido. Una transformación nueva se incorpora como un paso adicional del pipeline, no como código independiente.

10. Defectos frecuentes

Defecto	Corrección
Notebook con numeración discontinua o sin salidas	Reiniciar el núcleo y ejecutar todo antes del último commit
El informe afirma lo que el repositorio no evidencia	Contrastar cada afirmación con el artefacto versionado
Carpeta <code>src/</code> declarada y vacía	Trasladar el código fuera de las celdas antes de redactar
Identidades de Git duplicadas	Ejecutar <code>git shortlog -sne</code> y unificar nombre y correo
Omitir <code>self</code> o escribir <code>__init__</code> con un guion bajo	Si el intérprete reclama un argumento de más, falta <code>self</code>
Mutar el DataFrame recibido	Aplicar <code>df.copy()</code> antes de cualquier modificación
Definir clases sin estado	Si no hay estado entre invocaciones, corresponde una función
Omitir <code>super().__init__()</code>	Invocarlo como primera línea del constructor derivado

11. Verificación previa a la entrega

Notebook

- Ejecución completa tras reiniciar el núcleo
- Numeración continua desde el 1
- Salidas conservadas
- Mediciones dentro del cuaderno
- Celda narrativa antes de cada bloque

Código

- Clase base y al menos dos derivadas
- Cada principio localizable en el código
- El pipeline no consulta tipos concretos
- Dos implementaciones comparadas con su cifra
- Equivalencia funcional verificada

Repositorio

- Carpetas F2 y F3 diferenciadas
- README con ejecución y dependencias
- Commits de todos los integrantes
- Identidades de Git unificadas

Informe

- Portada con el rótulo de la fase
- Índice actualizado
- Todos los apartados de la plantilla
- El patrón nombrado y justificado
- Cinco referencias en APA 7 citadas en el texto

12. Referencias

Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object oriented software*. Addison Wesley.

Grus, J. (2019). *Data science from scratch* (2.ª ed.). O'Reilly Media.

McKinney, W. (2022). *Python for data analysis* (3.ª ed.). O'Reilly Media.

Python Software Foundation. (s. f.). *Classes*. <https://docs.python.org/3/tutorial/classes.html>

Python Software Foundation. (s. f.). *timeit: Measure execution time of small code snippets*. <https://docs.python.org/3/library/timeit.html>